Week 9: Deploy a risk agent to AWS and a simulated on-prem host

This PDF follows the lab order. Sections 1 through 6 are the evidence: IAM policies, the ECR push, the Fargate task, networking, the Ollama client, and the local run of case 900-01-0001. The written deployment document and the cost, complexity, and quality comparison follow. Console screenshots are inserted when the PNG files are present in submission/screenshots/. The click path is docs/CONSOLE.md.

1. IAM role and attached policy

Task role underwriting-risk-task-role is what the container uses. Execution role underwriting-risk-execution-role is what ECS uses to pull the image and write logs. Trust policy for both roles:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ecs-tasks.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Task role inline policy. Replace REGION and ACCOUNT before saving it in the console.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InvokeNovaLite",
      "Effect": "Allow",
      "Action": "bedrock:InvokeModel",
      "Resource": [
        "arn:aws:bedrock:REGION:ACCOUNT:inference-profile/us.amazon.nova-lite-v1:0",
        "arn:aws:bedrock:*::foundation-model/amazon.nova-lite-v1:0"
      ]
    }
  ]
}
```

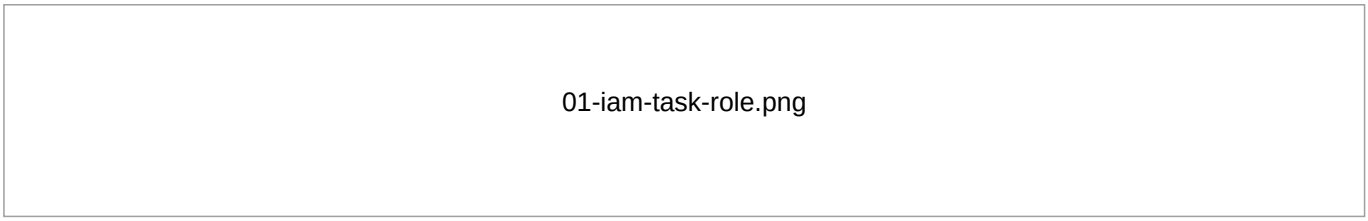
Execution role inline policy. Same REGION and ACCOUNT replacement.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "EcrAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "EcrPullThisRepo",
      "Effect": "Allow",
```

```
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage"
      ],
      "Resource": "arn:aws:ecr:REGION:ACCOUNT:repository/underwriting-risk-agent"
    },
    {
      "Sid": "WriteThisLogGroup",
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": [
        "arn:aws:logs:REGION:ACCOUNT:log-group:/ecs/underwriting-risk-agent",
        "arn:aws:logs:REGION:ACCOUNT:log-group:/ecs/underwriting-risk-agent:*"
      ]
    }
  ]
}
```

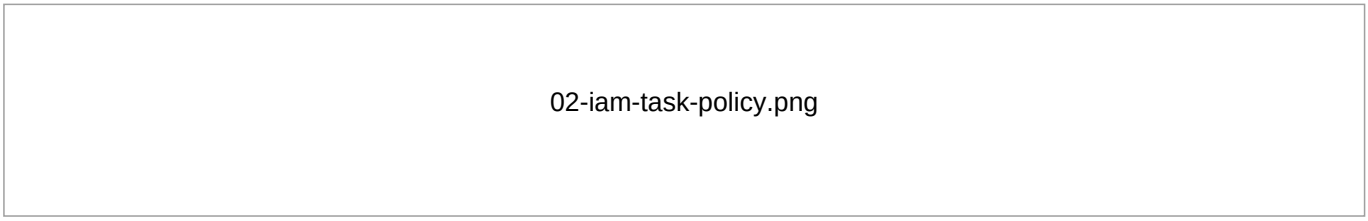
IAM task role underwriting-risk-task-role. Show the trust relationship.

Save this console screenshot as submission/screenshots/01-iam-task-role.png and rerun python3 submission/build_pdf.py.



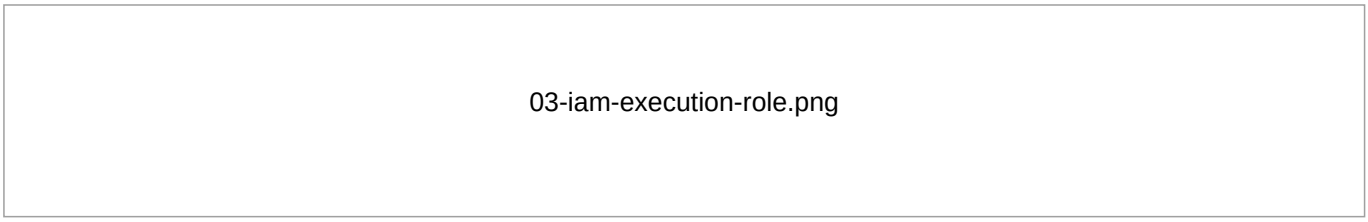
Inline policy InvokeNovaLite. Both Bedrock ARNs must be readable.

Save this console screenshot as submission/screenshots/02-iam-task-policy.png and rerun python3 submission/build_pdf.py.



IAM execution role underwriting-risk-execution-role.

Save this console screenshot as submission/screenshots/03-iam-execution-role.png and rerun python3 submission/build_pdf.py.



Inline policy PullRepoAndWriteLogs. Repository ARN and log group ARN readable.

Save this console screenshot as submission/screenshots/04-iam-execution-policy.png and rerun python3 submission/build_pdf.py.

04-iam-execution-policy.png

2. Build, push, and deploy

From the repo root, after ECR shows View push commands. Replace ACCOUNT.

```
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin \
  ACCOUNT.dkr.ecr.us-east-1.amazonaws.com
docker build -t underwriting-risk-agent .
docker tag underwriting-risk-agent:latest \
  ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/underwriting-risk-agent:latest
docker push ACCOUNT.dkr.ecr.us-east-1.amazonaws.com/underwriting-risk-agent:latest
```

Then ECS, task definition underwriting-risk-agent, Deploy, Run task (not Create service). Cluster underwriting-risk, the public subnet, security group underwriting-risk-sg, public IP turned on. The container has no port mapping. Environment: LLM_PROVIDER=bedrock, BEDROCK_MODEL_ID=us.amazon.nova-lite-v1:0, AWS_REGION=us-east-1. Task role and execution role are the two roles above.

Terminal: docker build, docker tag, and docker push to ECR.

Save this console screenshot as submission/screenshots/05-build-push.png and rerun python3 submission/build_pdf.py.

05-build-push.png

3. AWS task output

A correct CloudWatch log is one JSON object. deterministic_score is 20 and deterministic_band is low for Priya Shah. llm_score is an integer from 0 to 100. agreed is true when the model band is also low, and the kept score is the higher of the two. The task status is Stopped and the container exit code is 0.

ECS task stopped with container exit code 0.

Save this console screenshot as submission/screenshots/06-task-stopped.png and rerun python3 submission/build_pdf.py.

06-task-stopped.png

CloudWatch log. deterministic_score 20 and deterministic_band low.

Save this console screenshot as submission/screenshots/07-cloudwatch-output.png and rerun python3 submission/build_pdf.py.

07-cloudwatch-output.png

4. Networking

Default VPC, one public subnet (auto-assign public IPv4 on), public IP on at Run task. Security group underwriting-risk-sg has no inbound rules. The only outbound rule is TCP 443 to 0.0.0.0/0. The task does not listen. The address is for outbound calls to ECR, CloudWatch Logs, and Bedrock. A private subnet would need a NAT gateway or interface endpoints, which this one-shot task does not use.

Public subnet in the default VPC. Auto-assign public IPv4 is Yes.

Save this console screenshot as submission/screenshots/08-subnet.png and rerun python3 submission/build_pdf.py.

08-subnet.png

Security group underwriting-risk-sg. No inbound rules. Outbound TCP 443 only.

Save this console screenshot as submission/screenshots/09-security-group.png and rerun python3 submission/build_pdf.py.

09-security-group.png

5. Ollama setup and the local client

Ollama is installed on this machine. The on-prem model is qwen3:8b (about 5.2 GB), with think set to false. The on-prem path does not call Bedrock. LLM_PROVIDER=ollama selects risk_agent/llm_ollama.py, which posts to http://127.0.0.1:11434/api/chat.

```
"""Local Ollama client. The model runs on this machine and the prompt does not leave it."""

from __future__ import annotations

import json
import os
import urllib.error
import urllib.request

DEFAULT_HOST = "http://127.0.0.1:11434"
DEFAULT_MODEL = "qwen3:8b"
```

```
class OllamaError(Exception):
    pass


class OllamaClient:
    def __init__(self, model: str = None, host: str = None, http_timeout: float = 180) -> No
ne:
        self.model = model or os.environ.get("OLLAMA_MODEL", DEFAULT_MODEL)
        self.host = (host or os.environ.get("OLLAMA_HOST", DEFAULT_HOST)).rstrip("/")
        self.http_timeout = http_timeout

    def chat(self, system: str, user: str) -> tuple:
        payload = {
            "model": self.model,
            "stream": False,
            "think": False,
            "format": "json",
            "messages": [
                {"role": "system", "content": system},
                {"role": "user", "content": user},
            ],
            "options": {"temperature": 0},
        }
        body = _post_json(self.host + "/api/chat", payload, self.http_timeout)
        message = body.get("message") or {}
        content = message.get("content") or ""
        prompt_tokens = int(body.get("prompt_eval_count") or 0)
        completion_tokens = int(body.get("eval_count") or 0)
        return content, prompt_tokens, completion_tokens


def _post_json(url: str, payload: dict, timeout: float) -> dict:
    data = json.dumps(payload).encode("utf-8")
    request = urllib.request.Request(
        url,
        data=data,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    try:
        with urllib.request.urlopen(request, timeout=timeout) as response:
            return json.loads(response.read().decode("utf-8"))
    except urllib.error.HTTPError as exc:
        detail = exc.read().decode("utf-8", errors="replace")
        raise OllamaError("ollama HTTP {0}: {1}".format(exc.code, detail)) from exc
```

6. Simulated on-prem output

Command: LLM_PROVIDER=ollama OLLAMA_MODEL=qwen3:8b python3 -m risk_agent

Same fixture, fixtures/priya_shah.json. stdout:

```
{
  "score": 20,
  "band": "low",
  "factors": [
    "No prior claims",
    "Excellent credit",
    "Low-risk occupation",
```

```
    "Non-smoker",
    "Not in flood zone"
  ],
  "llm_score": 15,
  "llm_band": "low",
  "deterministic_score": 20,
  "deterministic_band": "low",
  "agreed": true,
  "prompt_tokens": 368,
  "completion_tokens": 48
}
```

deterministic_score is 20 and deterministic_band is low. qwen3:8b with thinking off returned llm_score 15, also low, so agreed is true and the kept score is 20. The factors match the application. That 15 matches the Week 8 score for this applicant.

7 and 8. Deployment document, comparison, and recommendation

Deployment, networking, and access control

The agent is the Week 8 risk scorer, deployed on its own. It reads one intake and one enrichment payload, asks a model for a score, and computes a second score in code from the bureau row. The test case is Priya Shah, case `900-01-0001`, in `fixtures/priya_shah.json`: a 31-year-old librarian in Vermont, $75,000 renters, no claims, excellent credit, low-risk occupation, not in a flood zone.

The code score uses the Week 8 point table: base 20, plus 15 per prior claim, 0 to 35 for the credit band, 0 to 25 for the occupation class, and 25 for a flood zone, capped at 100. Low is under 35, medium is 35 through 74, and high is 75 and above. For this bureau row the code score is 20 and the band is low. If the model's band matches, the case keeps the higher of the two scores. If the bands differ, `score` and `band` are null and `agreed` is false.

IAM

Two roles, both trusted only by `ecs-tasks.amazonaws.com`. The console login that creates them is the deployer. It is not attached to the task.

Task role: `underwriting-risk-task-role`

The process inside the container uses this role. boto3 finds it from the task metadata. The image contains no access keys.

Inline policy `InvokeNovaLite` allows one action:

- `bedrock:InvokeModel` on `arn:aws:bedrock:us-east-1:ACCOUNT:inference-profile/us.amazon.nova-lite-v1:0` and `arn:aws:bedrock:*::foundation-model/amazon.nova-lite-v1:0`.

`InvokeModel` is the permission the Bedrock Converse API checks. Nova Lite in us-east-1 is called through the `us.` inference profile, and that call also evaluates the foundation-model ARN, so both resources are listed. The policy has no other action. It cannot read S3, write logs, pull images, or call a second model. `ACCOUNT` is the 12-digit account id.

Execution role: `underwriting-risk-execution-role`

ECS itself assumes this role before the application starts. The application code never uses it.

Inline policy `PullRepoAndWriteLogs`:

- `ecr:GetAuthorizationToken` on `*`. This action has no resource-level ARN. It is the login step that lets Fargate pull from a private repository.
- `ecr:BatchCheckLayerAvailability`, `ecr:GetDownloadUrlForLayer`, and `ecr:BatchGetImage` on `arn:aws:ecr:us-east-1:ACCOUNT:repository/underwriting-risk-agent` only. Those three calls download this image. The role cannot pull any other repository.
- `logs:CreateLogStream` and `logs:PutLogEvents` on the log group `/ecs/underwriting-risk-agent` and its streams only. The group is created in the console first, so the role does not get `logs:CreateLogGroup`.

That split is the least-privilege cut. The role that can invoke the model cannot pull images. The role that can pull images cannot invoke the model.

Networking

Region us-east-1. Cluster `underwriting-risk`. The task definition is `underwriting-risk-agent` on Fargate, Linux/x86_64, 0.25 vCPU, 0.5 GB. The container `risk-agent` has no port mapping. It reads the baked-in fixture, calls Bedrock, prints JSON to stdout, and exits. There is no ECS service and no load balancer.

The task runs in the default VPC, in one subnet whose auto-assign public IPv4 setting is on, with **Public IP turned on** at Run task. Security group `underwriting-risk-sg` has no inbound rules. Its only outbound rule is TCP 443 to `0.0.0.0/0`.

The public address exists so the task can open outbound HTTPS to ECR (image pull), CloudWatch Logs, and Bedrock. Nothing accepts a connection. A private subnet would also keep the task off the inbound internet, and it would need a NAT gateway or interface endpoints for ECR, S3, Logs, and Bedrock. Those endpoints are billed every hour. This lab task has no clients, so the public subnet with an empty inbound list is the setup that matches the traffic.

What changes on the simulated on-prem path

The AWS container sets `LLM_PROVIDER=bedrock`. `risk_agent/llm_bedrock.py` calls `bedrock-runtime` `converse` on `us.amazon.nova-lite-v1:0`. The prompt leaves the task and is processed in Bedrock. Logs land in CloudWatch. The image is stored in ECR.

The on-prem run is the same `score_case` function with `LLM_PROVIDER=ollama`. `risk_agent/llm_ollama.py` posts to `http://127.0.0.1:11434/api/chat`. The model is `qwen3:8b`, already present on disk (about 5.2 GB). The request sets `think` to false so the reply is the JSON object, temperature 0, and `format` json. The prompt is handled by the local process. There is no task role, no ECR pull, and no CloudWatch stream.

A real air-gapped room breaks the AWS path immediately: no route to Bedrock, ECR, or CloudWatch, and no instance-metadata credentials. It also breaks a casual Ollama install. `ollama pull` needs the internet, so the weight file has to be copied in beforehand. Model updates, pip packages, and the container image move on media. This laptop is not that room. It is the same shape at small scale: the model that scores the applicant runs inside the machine that holds the application.

A client asks for that when the application text is not allowed to leave their building. A private subnet in front of Bedrock still sends the prompt to Amazon. Network isolation and data residency are different requirements. The private subnet meets the first. Only the local model meets the second.

Comparison and recommendation

Cost

The Fargate task is 0.25 vCPU and 0.5 GB, and it exits when the print finishes. At the public Fargate rates (about $0.04048 per vCPU-hour and $0.004445 per GB-hour in us-east-1) a one-minute task is a fraction of a cent, plus Nova Lite tokens for one short JSON call. The cost that matters is leaving something running: a service with desired count 1, or a NAT gateway at about $0.045 per hour plus data. This design uses neither.

The on-prem run has no per-token charge. The cost is the machine and the weight file already on it. At one case, that is cheaper. At a team's volume, the cost moves into hardware, patching, and someone who keeps the model process up.

Complexity

The AWS path is two IAM roles, a private ECR repository, a log group, one security group, a Fargate cluster, a task definition, Bedrock model access, and a docker push. After that, Run task is one action and the log is the result.

The on-prem path is Ollama, which was already installed, and one environment variable. The command is `LLM_PROVIDER=ollama OLLAMA_MODEL=qwen3:8b python3 -m risk_agent`. A production air-gapped copy of that path is the more complicated one, because nothing can be pulled on demand.

Output quality

Both paths score the same fixture. The code score does not depend on the model: `deterministic_score` 20, `deterministic_band` low.

The local `qwen3:8b` run on 13 Oct 2026, with thinking off, returned valid JSON on the first call (368 prompt tokens, 48 completion tokens, about 5 seconds):

- `llm_score` 15, `llm_band` low, `agreed` true
- kept `score` 20 and `band` low, because the bands match and the agent keeps the higher score
- factors: no prior claims, excellent credit, low-risk occupation, non-smoker, not in a flood zone

That 15 matches the Week 8 score for this same applicant. It sits in the low band, next to the code score of 20, which is a baseline with no extra risk added. An earlier run of `llama3.2:3b` on this same prompt returned 0. Same band, much weaker integer. `qwen3:8b` with thinking off is the on-prem result.

The AWS task uses Amazon Nova Lite through Bedrock. Confirm in the CloudWatch log that `deterministic_score` is 20, `deterministic_band` is low, and the model returned a score from 0 to 100. Put that `llm_score` next to the local 15. The band should still be low. If `agreed` is false, the model's band left the low range and the case would be escalated under the Week 8 rule.

Recommendation

For a client who requires the applicant data to stay inside their own boundary, use the Ollama deployment. The prompt is posted to 127.0.0.1 and the weights are on that machine. Fargate plus Bedrock, including a private subnet, still processes the text in AWS.

For a client who can send that text to a managed model, use the Fargate deployment. The task role can call only Nova Lite, the security group accepts no inbound traffic, and you do not operate the model host.

On this case the local `qwen3:8b` score of 15 already agrees with the code band. Run the AWS path as a task, not a service, so the bill stops when the container exits.